

1 Chp1

1.1 计算机图形学基本定义

- 图形系统由一台配备有快速处理器和大容量内存的主机组成。
 1. 显示设备: color monitor
 2. 输入设备: 键盘, 鼠标, 触控屏
 3. 输出设备: LCD panels 液晶面板, color printer, laser printer, plotter
 4. 接口设备:
- 图像类型 (根据图形的组成)
 1. raster graphics
 - 光栅图像 (也叫 bitmap image) 是由一系列像素组成的图像, JPEG, PNG, GIF
 2. vector graphics
 - mathematical formulas is used for plotting different types of shapes.
- 图像类型 (根据图形的控制)
 1. Interactive graphics
 - 在可交互图像中, 用户可以在图像上有一些控制, 并且可以对生成的图像做一些改变。
 2. Passive graphics
 - 在被动式图像中, 用户不可以对展示在显示屏上的图像进行一些控制。
- Application areas of Computer Graphics
 1. Computer-aided Design
 - Automobiles - wireframe outline shape of an automobiles internal components.
 - A multi-window environment with different orientation.

- Aircraft - wireframe outline for different features.
- Real-time Animation for testing performance.
- buildings.
- textiles.

2. Presentation Graphics

- Bar charts
- Pie charts
- Line graphs
- Surface graphs

3. Computer Art

- Cartoon drawing
- fine art(e-painting, photography)
- commercial art applications(advertising, logo)

4. Entertainments

- Movies
- Short films
- TV serials.

5. Education and Training

- Flight simulator
- Simulators for practice sessions.
- simulators of military tanks.

6. Image Processing

- CT scans.
- PS,PR application to improve the quality of a picture.

7. Graphical User Interface GUI

- Clickable buttons.
- Dialog box is to display additional information for users.
- Icon is a small symbol.
- Menu is a list of commands or choices.

8. Visualization

- Producing graphical representations for scientific, engineering datasets.
- Business visualization.

- 计算机图形系统关键组件

1. 输入设备

- Manual data entry devices: Manual input devices are those peripheral devices through which the user can enter the data manually at the time of processing. 键盘, 鼠标, joystick, touch screen
- Direct data entry devices: Direct data devices are those peripheral devices through which data is directly feed from the source to computer system. Scanner, Barcode reader, OCR

2. 输出设备

- impact printer 击打式打印机, 击打式打印机有一个打印头, 打印头会撞击装满墨水的色带, 色带被打印头压在纸上, 字就印在了纸上。鼓式打印机, 菊花轮打印机, 点阵打印机。
- Non-impact printer 非击打式打印机, 非击打式打印机使用喷头将墨水喷在纸上, 形成文字和图案。喷墨打印机, 激光打印机。
- plotter 绘图仪 o、o

3. 显示设备

显示设备也是输出设备用于以视觉形式呈现信息。Display System 也被称为 video monitor/ video display unit。CRT, LED, LCD, DVST。

- Cathode ray tube(CRT, 阴极射线管)

1. Components

- Electron gun 电子枪, 主要由 heating filament 加热灯丝/heater 组成。The electron gun emits the ray of the electrons.

- Focusing and accelerating anodes 聚焦和加速阳极, 聚焦和加速阳极将电子枪产生的电子, make a narrow and shapely beam of electrons.
 - Horizontal and vertical deflection plates 水平和垂直偏转板, 用于引导 the path of electron beam 电子束的路径. Deflection plates produces an electromagnetic field that bends electron beam when traveling the area.
 - Phosphorus-coated screen 涂磷屏幕。涂磷屏幕用于在高速撞击时产生亮点。The phosphorus coated screen is used to produce bright spots when the high-velocity electron beam hits it.
2. Raster Scan 光栅扫描是一种电子束在屏幕上从上到下一行一行扫描形成图像的技术。
 3. Random Scan 随机扫描, 电子束直接指向需要绘制图像的区域, 并且绘制线条图像。
- Liquid crystal display(LCD)
 1. LCD is a flat panel display technology, which uses liquid crystal to turn pixels on or off and when the current inside through it, its position changes desired color.
 - Light emitting diode(LED)
 1. light emitting diode is a semiconductor device which emits light when current passes through it.
 - Direct view storage tube(DVST 直观存储管): DVST 主要将图像信息存储为 phosphorus-coated 屏幕后的 charge distribution.
 1. Components
 - Primary gun is used to store picture information.
 - Secondary gun is used to display a picture on the screen.
 2. Advan
 - (a) Less time consuming

(b) High resolution

(c) Less cost

2 Chp2

2.1 Color Model

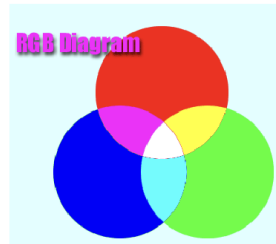
一个颜色模型是 hierarchical system 去通过一组原色集合生成所有色彩范围。

2.1.1 Additive Color Model

RGB Model, red, green, blue, 每一种原色都分配从 0-255 的数值。

2.1.2 Subjective Color Model

CMYK:Cyan, magenta, yellow, black. is used for printer.

RGB Diagram

Basis colors: R, G, B

R: Red=[1, 0, 0]

G: Green=[0, 1, 0]

B: Blue=[0, 0, 1]

C: Cyan=[0, 1, 1]

M: Magenta=[1, 0, 1]

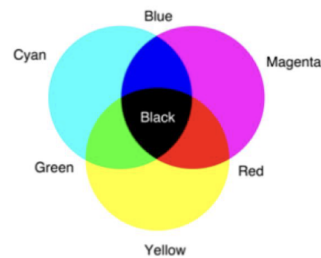
Y: Yellow=[1, 1, 0]

W: White=[1, 1, 1]

1-5

Computer Graphics

Color Models in Computer Graphics

CMYK Diagram

Basis colors: C, M, Y

C: Cyan=[1, 0, 0]

M: Magenta=[0, 1, 0]

Y: Yellow=[0, 0, 1]

R: Red=[0, 1, 1]

G: Green=[1, 0, 1]

B: Blue=[1, 1, 0]

K: Black=[1, 1, 1]

图 1: ColorModel

2.2 图像表示

- 像素是图像的最小组成单位。
- The size of image 被定义为水平像素数量 * 垂直像素数量
- Aspect ratio of an image 图像的纵横比是高度与图像宽度比。
- 图像分辨率是一个单位的像素数量。
- JPEG, PNG, TIFF, GIF

2.2.1 2-Dimention and 3-Dimention

- 2-Dimention

1. 2 维图像由一个有两个维度长度和宽度的平面进行表示。常常用于 traditional printing, typography, drawing.

- 3-Dimention

1. 3 维度图像由 3 个维度 length width height 进行表示。
2. 3 维图像是几何数据的表示

- 2D vs 3D

2-D	3-D
A. In 2-D, image is comprised by height, weight.	A. In 3-D, Image is comprised by height, weight, depth.
B. In 2-D, image is in mathematical form , x-axis, y-axis.	B. In 3-D, image is in mathematical form x-axis, y-axis, z-axis.
C. Geometry in 2-D are Rectangle, square, triangle, polygon etc.	C. Geometry in 3-D are cylinder, cube, spear.
D. It is not suitable for conceptual drawing.	D. It is suitable for conceptual drawing.
E. In 2-D, images is created by the traditional drawing method.	E. In 3-D, everything is created by computer software.

图 2: 2Dvs3D

2.2.2 2-Dimention

变换包括 translation, rotation, reflection, scaling.

- 2D Translation 移动一个物体从一个位置到另一个位置。

Translation distance pair (T_x, T_y) is called translation vector/shift vector.

We can also represent the translation in matrix form-

$$\begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} T_x \\ T_y \end{bmatrix} + \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}$$

图 3: 2DTranslation

- 2D Scaling 在二维平面上调整物体大小
[Scaling of an Object= Original co-ordinates * Scaling Factors]
- 2D Rotation 将物体从原点旋转到特定角度，
旋转需要旋转点 (pivot point), 旋转角 θ , 旋转有两种方式 clockwise(顺时针旋转, 负的) 和 anti-clockwise(逆时针旋转, 正的)。

Computer Graphics

2D Rotation

Example-

Rotation of a Point: If we want to Rotate a point A (P_0, Q_0) that has an original Rotation angle (θ) with distance r from origin to A' (P_1, Q_1) that has a Rotation angle β .

We can also define point A' (P_1, Q_1) in the same way- Using trigonometry equations

$$P_1 = r \cos(\theta + \beta) = r \cos \theta \cos \beta - r \sin \theta \sin \beta \dots\dots\dots (3)$$

$$Q_1 = r \sin(\theta + \beta) = r \cos \theta \sin \beta + r \sin \theta \cos \beta \dots\dots\dots (4)$$

$r \cos \theta$ and $r \sin \theta$ can be replaced here using equation no. (1) and (2)

$$P_0 = r \cos \theta \dots\dots\dots (1)$$

$$Q_0 = r \sin \theta \dots\dots\dots (2)$$

1-37

Computer Graphics

2D Rotation

Example-

Rotation of a Point: If we want to Rotate a point A (P_0, Q_0) that has an original Rotation angle (θ) with distance r from origin to A' (P_1, Q_1) that has a Rotation angle β .

By substituting expressions in (1) (2) (3) (4), we obtain the equations for rotating a point at position A (P_0, Q_0) through an angle β about the origin

$$P_1 = P_0 \cos \beta - Q_0 \sin \beta$$

$$Q_1 = P_0 \sin \beta + Q_0 \cos \beta$$

图 4: 2DRotation-induce

Rotation of a Point: If we want to Rotate a point A (P_0, Q_0) that has an original Rotation angle (θ) with distance r from origin to A' (P_1, Q_1) that has a Rotation angle β

We can also represent the Rotation in the form of matrix-

$$\begin{bmatrix} P_1 \\ Q_1 \end{bmatrix} = \begin{bmatrix} \cos \beta & -\sin \beta \\ \sin \beta & \cos \beta \end{bmatrix} \times \begin{bmatrix} P_0 \\ Q_0 \end{bmatrix}$$

图 5: 2DRotation

- 2D Reflection Producing a mirror image of an object is called reflection. 产生物体的镜像被称作反射。二维反射相对于一个轴生成镜像通

过将物体绕着这个轴旋转 180 度。

1. 绕 x 轴反射

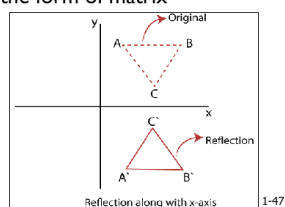
We can represent the Reflection along x-axis by following equation-

$$X_1 = X_0$$

$$Y_1 = -Y_0$$

We can also represent Reflection in the form of matrix-

$$\begin{pmatrix} X_1 \\ Y_1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \times \begin{pmatrix} X_0 \\ Y_0 \end{pmatrix}$$



Computer Graphics

2D Reflection

1. Reflection along X-axis:

Homogeneous Coordinate Representation:

We can also represent the Reflection along x-axis in the form of 3 x 3 matrix

$$\begin{pmatrix} X_1 \\ Y_1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} X_0 \\ Y_0 \\ 1 \end{pmatrix}$$

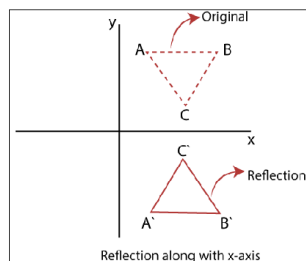


图 6: X-axis-Reflect

2. 绕 y 轴反射

Homogeneous Coordinate Representation:

We can also represent the Reflection along y-axis in the form of 3 x 3 matrix

$$X_1 = -X_0$$

$$Y_1 = Y_0$$

$$1 = 1$$

$$\begin{pmatrix} X_1 \\ Y_1 \\ 1 \end{pmatrix} = \begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} X_0 \\ Y_0 \\ 1 \end{pmatrix}$$

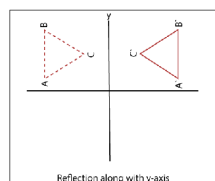


图 7: Y-axis-Reflect

3. 绕垂直于 XY 的平面进行反射，也就是绕着原点旋转 180 度

$$\begin{aligned} X_1 &= -X_0 \\ Y_1 &= -Y_0 \\ 1 &= 1 \end{aligned}$$

$$\begin{pmatrix} \mathbf{X}_1 \\ \mathbf{Y}_1 \\ \mathbf{1} \end{pmatrix} = \begin{pmatrix} -\mathbf{1} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & -\mathbf{1} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{1} \end{pmatrix} \mathbf{x} \begin{pmatrix} \mathbf{X}_0 \\ \mathbf{Y}_0 \\ \mathbf{1} \end{pmatrix}$$

图 8: PerpendicularXYPlant-Reflect

4. 沿 y=x 进行反射

$$\begin{pmatrix} \mathbf{X}_1 \\ \mathbf{Y}_1 \\ \mathbf{1} \end{pmatrix} = \begin{pmatrix} \mathbf{0} & \mathbf{1} & \mathbf{0} \\ \mathbf{1} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{1} \end{pmatrix} \mathbf{x} \begin{pmatrix} \mathbf{X}_0 \\ \mathbf{Y}_0 \\ \mathbf{1} \end{pmatrix}$$

图 9: YX-Reflection

2.2.3 3-Dimention Transformation

- 3D Translation

- Here, the translation coordinates i.e (T_x, T_y, T_z) are also called the translation or shift Vector.

$$\begin{aligned}x_1 &= x_0 + T_x, \\y_1 &= y_0 + T_y, \\z_1 &= z_0 + T_z\end{aligned}$$

- We can also represent the 3D Translation in a matrix form provided below-

$$\begin{pmatrix} x_1 \\ y_1 \\ z_1 \end{pmatrix} = \begin{pmatrix} T_x + x_0 \\ T_y + y_0 \\ T_z + z_0 \end{pmatrix}$$

图 10: 3DTranslation

• 3D Scaling

The initial coordinates of object = P (x_0, y_0, z_0)

Scaling factor for x-axis = S_x

Scaling factor for y-axis = S_y

Scaling factor for z-axis = S_z

The coordinates after Scaling = Q (x_1, y_1, z_1)

Equation of Scaling $Q = P * S$

We can represent the 3D Scaling in the form of equation-

$$x_1 = x_0 \cdot S_x$$

$$y_1 = y_0 \cdot S_y$$

$$z_1 = z_0 \cdot S_z$$

图 11: 3DScaling

• 3D Rotation

1. X axis Rotation

X-axis Rotation:

We can represent 3D rotation in the form of matrix-

$$\begin{pmatrix} x_1 \\ y_1 \\ z_1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_0 \\ y_0 \\ z_0 \\ 1 \end{pmatrix}$$

图 12: 3DX-Rotate

2. Y axis Rotation

$$\begin{pmatrix} \mathbf{x}_1 \\ \mathbf{y}_1 \\ \mathbf{z}_1 \\ \mathbf{1} \end{pmatrix} = \begin{pmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \mathbf{x} \begin{pmatrix} \mathbf{x}_0 \\ \mathbf{y}_0 \\ \mathbf{z}_0 \\ \mathbf{1} \end{pmatrix}$$

图 13: 3DY-Rotate

3. Z axis Rotation

$$\begin{pmatrix} \mathbf{x}_1 \\ \mathbf{y}_1 \\ \mathbf{z}_1 \\ \mathbf{1} \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \mathbf{x} \begin{pmatrix} \mathbf{x}_0 \\ \mathbf{y}_0 \\ \mathbf{z}_0 \\ \mathbf{1} \end{pmatrix}$$

图 14: 3DZ-Rotate

• 3D Reflection

1. XY Plane reflection

- The initial coordinates are = (x_0, y_0, z_0)
- The new coordinates are = (x_1, y_1, z_1)

$$x_1 = x_0$$

$$y_1 = y_0$$

$$z_1 = -z_0$$

Matrix of 3D Reflection-

$$\begin{pmatrix} \mathbf{x}_1 \\ \mathbf{y}_1 \\ \mathbf{z}_1 \\ \mathbf{1} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \mathbf{x} \begin{pmatrix} \mathbf{x}_0 \\ \mathbf{y}_0 \\ \mathbf{z}_0 \\ \mathbf{1} \end{pmatrix}$$

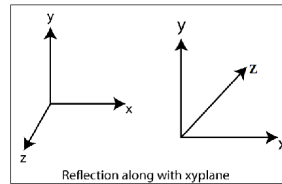


图 15: 3DXYPlane-Reflection

2. XZ Plane reflection

- The initial coordinates are = (x_0, y_0, z_0)
- The new coordinates are = (x_1, y_1, z_1)

$$x_1 = x_0$$

$$y_1 = -y_0$$

$$z_1 = z_0$$

Matrix of 3D Reflection-

$$\begin{bmatrix} x_1 \\ y_1 \\ z_1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

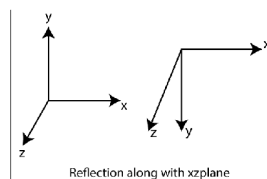


图 16: 3DXZPlane-Reflection

3. YZ Plane reflection

$$x_1 = -x_0$$

$$y_1 = y_0$$

$$z_1 = z_0$$

Matrix of 3D Reflection-

$$\begin{bmatrix} x_1 \\ y_1 \\ z_1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

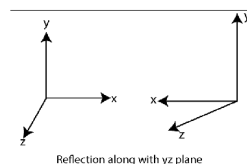


图 17: 3DYZPlane-Reflection

3 Chp3

3.1 Output Primitive 输出图元

输出图元是生成不同计算机图像对象的简单几何架构。

- Point

点是最基本的图像表示元素，是线的最小单位。

- Line

线被定义为两个 end point 的连接

- Polyline 折线

折线是线段序列的连接，由 N-1 条线连接 N 个点，并且第一个点与最后一个点不连接，折线中的每一条线都被称为 edge。

- polygon 多边形

多边形是一种闭合的图元，其使用 N 条线连接 N 个点，第一个点与最后一个点相连接。

- Filled Region 填充区域

Filled region 是一个帮助我们填充一个对象、区域或者图像的方法。

3.2 Filled Algorithm 填充算法

- Seed Fill Algorithm 种子填充算法

在种子填充算法中我们会在多边形的边界内选取一个起点叫做 seed。

1. Flood-fill algorithm

Flood-fill algorithm colors whole area through interconnected pixels by single color. 类似于 paint program 中的 bucket tools。

2. Boundary-fill algorithm

先通过颜色或某个属性轮廓确定图像对象边界，在边界内部任意种子点出发，只在边界所包围的区域填充颜色。

- Scan Line Algorithm 扫描线算法

扫描线从上到下扫描图像，每次扫描线与图像多边形边界产生交点，将这些交点按照 x 坐标排序，将排序好的线段两两组成一个线段，并且使用对应颜色填充线段。

3.3 Drawing Line Algorithm 直线绘制算法

$y = mx + c$, m 是斜率, c 是截距。 $m = \frac{x_1 - x_2}{y_1 - y_2}, c = y - mx$

- Digital differential analyzer 数字差分分析仪

首先，我们知道两个点 起点 (x_0, y_0) , 终点 (x_k, y_k) , 因此对于 $m = \frac{y_k - y_0}{x_k - x_0}$ 我们分为 3 种情况：

Case1: 如果 $m < 1$, 从 k 到 $k+1$ 在 x 轴的改变量大于在 y 轴的改变量，因此设置 $x_{k+1} - x_k = 1$ ，此时根据斜率计算公式， $y_{k+1} - y_k = m$ 。

Case2: 如果 $m > 1$, 从 k 到 $k+1$ 在 y 轴的改变量大于在 x 轴的改变量，因此设置 $y_{k+1} - y_k = 1$ ，此时根据斜率计算公式， $x_{k+1} - x_k = \frac{1}{m}$ 。

Case3: 如果 $m=1$, 从 k 到 $k+1$ 在 y 轴的改变量等于在 x 轴的改变量, 因此设置 $x_{k+1} - x_k = 1$, $y_{k+1} - y_k = 1$ 。

- Bresenham's Line Drawing 布雷森汉姆直线绘制算法

首先, 我们知道两个点 起点 (x_0, y_0) , 终点 (x_k, y_k) , 因此对于 $\Delta y = y_k - y_0$, $\Delta x = x_k - x_0$, $m = \frac{\Delta y}{\Delta x}$

计算 decision parameter $p_k = 2\Delta y - \Delta x$.

接下来分为 2 种情况:

Case1: 如果 $p_k < 0$, 那么 $x_{k+1} - x_k = 1, y_{k+1} = y_k, p_{k+1} = p_k + 2\Delta y$

Case2: 如果 $p_k \geq 0$, 那么 $x_{k+1} - x_k = 1, y_{k+1} - y_k = 1, p_{k+1} = p_k + 2\Delta y - 2\Delta x$,

- Mid point line Drawing 中点画线算法

首先, 我们知道两个点 起点 (x_0, y_0) , 终点 (x_k, y_k) , 因此对于 $\Delta y = y_k - y_0$, $\Delta x = x_k - x_0$, $\Delta d = 2(\Delta y - \Delta x)$

计算 decision paramete $d_i = 2\Delta y - \Delta x$

2 种情况:

Case1: 如果 $d_i < 0$, 那么 $x_{k+1} - x_k = 1, y_{k+1} = y_k, d_{i+1} = d_i + 2\Delta y$

Case2: 如果 $d_i \geq 0$, 那么 $x_{k+1} - x_k = 1, y_{k+1} - y_k = 1, d_{i+1} = d_i + 2\Delta d$

4 Chp4

4.1 Clipping 剪裁

Clipping is a procedure that identifies which portions of a picture that are either inside or outside of our viewing pane.

选择用于显示图像的区域被称为窗口/剪裁窗口。

显示设备上用于映射窗口的区域被称为视图。

我们定义要显示图像所处的坐标系被叫作世界坐标系。

4.2 Point Clipping 点剪裁

点剪裁是确定点在视口内部或者视口外部的点过程。

假设我们有一个视口, X 的范围 (xw_{min}, xw_{max}) , Y 的范围 (yw_{min}, yw_{max}) , 我们现在有一个点 $A(p, q)$, 如何判断 A 是否在窗口里面?

如果 $xw_{min} \leq p \leq xw_{max}$ and $yw_{min} \leq q \leq yw_{max}$, 那么点就落在视口之中, 如果不满足以上条件, 那么点就需要 clipping。

4.3 Line Clipping 线段剪裁

线段剪裁是我们可以裁剪掉视口外部的线段, 保留视口内部的线段。

一般过程: 首先判断给定的线段是否完全在线段内部; 如果不是的话, 判断线段是否完全在视口外部; 如果无法确定一条直线完全在内部或者外部, 我们要计算一个或者多个裁剪边界的交点。

4.3.1 Cohen Sutherland Line Clipping Algorithm

将 view panel 视角分为 9 个部分, 从 1 到 9 是从左上到右下, 使用 4 位二进制数对这 9 个部分进行表示, 4 位分别对应着上下右左。9 个区域的区域 5 也就是中间是 0000 表示线段在这里面。

eg1: 如果一条线的端点出现在右上角也就是区域 3, 那么对应的数字为 1010。

- 算法流程

1. 给线段的两个端点分配 bit code
2. 对两个端点的 bit code 做 OR 操作, 如果结果为 0000, 那么线段在 0000 区域中, 不用裁剪。
3. 否则对两个端点取 AND 操作,
4. 如果 AND 操作结果为 0000, 那么部分线段在视口区域中
5. 否则, 线段不再视口区域中
6. 如果一条线需要被剪裁, 需要找到这条线与 4 个边界的交点
7. 如果这条线与左边界有交点, 我们已知直线的斜率, 并且已知左边界点的 xw_{min} , 我们可以通过斜率计算公式求出左边界点的 y 值。 $y_0 = y_1 + m(x_0 - x_1)$ 。
8. 如果这条线与右边界有交点, 同理, $y_0 = y_1 + m(x_0 - x_1)$ 。
9. 如果这条线与上边界有交点, $x_0 = x_1 + (y_0 - y_1)/m$
10. 如果与下边界有交点, $x_0 = x_1 + (y_0 - y_1)/m$

4.3.2 Liang-Barsky Line Clipping Algorithm

- 算法流程

1. 直线有两个端点, (x_1, y_1) and (x_2, y_2) , $\Delta x = x_2 - x_1$, $\Delta y = y_2 - y_1$ 。
 $\Delta x > 0$ 表示直线从左向右延伸, $\Delta y > 0$ 表示直线从下向上延伸
2. 计算方向参数 p_k , 直线相对于左边界水平方向 $p_1 = -\Delta x$ 若 $p_1 < 0$ 那么直线由外向内走; 直线相对右边界的水平方向 $p_2 = \Delta x$; 直线相对下边界的垂直方向 $p_3 = -\Delta y$, 直线相对上边界的垂直方向 $p_4 = \Delta y$
3. 计算距离参数 q_k , 起点到左边界的水平距离 $q_1 = x_1 - xw_{min}$, 起点到右边界的水平距离 $q_2 = xw_{max} - x_1$, 起点到上边界的垂直距离 $q_3 = yw_{max} - y_1$, 起点到下边界的垂直距离 $q_4 = y_1 - yw_{min}$
4. 更新 t 的数值, t 是 time interval, 开始 $t_1 = 0$, 结束 $t_2 = 1$, 从起点到终点。我们的坐标公式 $x = x + t\Delta x$, $y = y + t\Delta y$
5. 如果, 我们遍历 p_k , 如果 $p_k = 0$, 这一条直线与某一条边界平行, 并且如果对应的 $q_k < 0$, 那么这条平行线段在视口之外。
6. 如果 $p_k < 0$ 线段从视口外向内走, $t_1 = \max(t_1, q_k/p_k)$, q_k/p_k 计算进入时间。
7. 如果 $p_k > 0$, $t_2 = \min(t_2, q_k/p_k)$, q_k/p_k 计算离开时间从窗口内走出的时间。
8. 如果 $t_1 > t_2$, 进入时间大于离开时间, 那么这条线不可视
9. 否则 $t_1 < t_2$, 计算剪裁后的端点; 如果 $t_1 > 0$, $x = x + t_1\Delta x$, $y = y + t_1\Delta y$, 如果 $t_2 < 1$, $x = x + t_2\Delta x$, $y = y + t_2\Delta y$, 更新起点和终点。

4.3.3 Midpoint Subdivision Line Clipping Algorithm 中点割线裁剪算法

- 算法流程

1. 像 Cohen Sutherland Clipping Algorithm 为线段两个端点分配 bit code。

2. 两个端点的 bit code 实现 OR 操作, 如果 OR 操作结果为 0000, 则线段在视口内;
3. 否则, 对两个端点实现 AND 操作, 如果 AND 操作不为 0000, 则线段在视口外不可见, 如果 AND 为 0000, 则线段部分在视口内部分在视口外
4. 对于部分可见的线段, 使用 $X_m = (x_1 + x_2)/2, Y_m = (y_1 + y_2)/2$ 求线段中点。要知道两个端点在窗口内还是窗口外。我们求出的中点如果在窗口内, 那么我们要把窗口内的端点更新到此中点; 如果求出来的中点在窗口外, 那么要把窗口外的端点更新到中点的位置。

4.4 Polygon Clipping 多边形剪裁

A polygon can be described as the enclosed collection of the lines.

4.4.1 Sutherland-Hodgeman polygon clipping algorithm

- 算法流程

1. 多边形剪裁算法要处理 4 种不同的剪裁情况, 包括左剪裁, 右剪裁, 上剪裁, 下剪裁。在对每一种情况进行剪裁时, 我们移除某一个部分在视口外的区域, 保留视口内的区域。比如在左剪裁中, 我们只移除多边形左侧位于窗口外部的部分, 保留窗口内部的部分。
2. 在做每一个 case 时, 多边形的每一条边 $v_1 = (x_1, y_1), v_2 = (x_2, y_2)$, 我们要考虑 4 个 conditions, 并且我们需要使用列表存储多边形的顶点和交点
3. 如果边是从外部到内部, 那么多边形的边和边界的交点记为 v'_1 , 列表中需要存储 v'_1 和 v_2
4. 如果边是从内部到内部, 那么列表要存储 v_2
5. 如果边是从内部到外部, 那么列表要存储 v'_1 ,
6. 如果边是从外部到外部, 则不用存储任何信息

4.4.2 Weiler-Atherton polygon clipping algorithm

- 算法流程
 1. 首先，创建一个用于标识进入窗口或者退出窗口状态的交点列表。
 2. 接着，创建一个包含 all the vertices of the polygon 在 subject polygon list；以及一个剪裁窗口点 Clip Polygon List，刚开始按顺时针存储着剪裁窗口的点。
 3. 遍历 subject polygon list 中的按顺序的两个点可以组成的边 E，并且遍历剪裁窗口中按顺序的两个点组成的边 C 是否与 E 相交，如果相交计算交点坐标，并且标记交点是入窗口还是出窗口以及其前驱和后继，并且将交点插入在 E 边和 C 边组成的交点之间。
 4. 从第一个进入点开始走，沿着主多边形列表走，直到遇到离开点，切换到剪裁窗口列表并且找到离开点，然后沿着剪裁窗口走，遇到进入点切换回主多边形列表，循环直到回到原点。

4.5 Text Clipping 文本剪裁

Text clipping is a process in which we remove the part of the string that is outside of the view pane.

4.5.1 All-or-none string-clipping 全有或者全无字符串剪裁

如果环绕在字符串矩形边界框与窗口边界有重叠，那么这个字符串直接被 rejected（直接消失），这也是最快的文本剪裁方法。

4.5.2 All-or-none character-clipping 全有或者全无字符剪裁

- Rejecting an entire character

单个字符的边界将会与窗口进行比较，如果字符边界与窗口边界重合或者位于窗口边界之外，此字符将会被剪裁。
- CLip the components of individual characters

如果单个字符与剪裁窗口重合，那么剪裁掉该字符位于窗口之外的部分。

一般来说如果是由线段构成的轮廓字符可以使用这种方法进行剪裁。

5 Chp5

5.1 Rendering

Rendering is an automatic process of creating digital image from a template by using software system.

- Rendering real time

Real-time rendering is a process where the image is produced and analyzed from 3D information at a fast speed.

- Render offline

Offline rendering is a technique mainly used in situations where the processing speed is slow.

5.1.1 Rendering techniques

- Z-Buffer

1. Z-Buffer is also called Depth Buffer, which is used for hidden surface detection. And Z-Buffer means a memory area that keeps z coordinates and intensity closest to the observers for each pixel.
2. Depth buffer requires 2 arrays. One is for keeping the intensity of the pixel, another one records z value.

- Scan line

1. Scan line is used for determining the visible surfaces one line at a time.
2. Scan line keeps the records of two edge lists. One is edge list which contains the coordinates of two endpoints of a edge. Other one is active edge line(AEL), it contains the edges that intersects with scan line during it sweeps.

- Hidden Line Rendering

Hidden line rendering is used to represent object whose surface is covered and blocked with lines representing the sides of the object.

- Ray tracing rendering

Ray tracing rendering follows the light on its way from the light source to the screen and estimates what kind of color is displayed on the pixel where the ray falls.

The ray tracing rendering builds an image by extending rays into a scene.

5.1.2 Features of rendering

- Shading 着色

Shading is an appearance of lights and darks found in a surface of an object.

- Texture mapping 纹理映射

Texture mapping is a graphics design procedure, that refers to the pattern of the surface.

- Bump mapping 凹凸映射

Bump mapping is a process that visualizing slight bumpiness on layers.

- Motion blur 运动模糊

Motion blur is caused by relative motion between image acquisition system and an object, and items appear blurry due to the increased movement or the movement of the camera.

5.2 Animation

Animation refers to any time sequence of visual changes in pictures, including size, shape, color, structure, angle.

5.2.1 Animation Function

- Morphing(变形)

变形包括 3 个过程:

1. One initial image and final image as two key frames are added the morphing application.
 2. The selection of key points on both the images for a smooth transition between two images.
 3. The key point of the first image transforms to the corresponding key points of the second image.
- Panning(平移)
 1. Panning refers to the movement of the fixed size window across the window object in a scene. And the direction in which the fixed size window moves, the window objects moves in opposite direction.
 - Zooming(缩放)
 1. In zooming, the window is fixed an object and changes its size. The objects also appear to change in size.
 2. Zooming In and Zooming Out are used to enlarge and reduce the size of the view.
 - Fractals(分形)
 1. Fractal function is used to generate the complex picture by the repetition of a single formulas again and again with a slightly different value.

5.3 Light Source

- Lighting model/ illumination model is used to calculate the intensity of light that is reflected at a given point on the surface.
- Lighting effect depends on 3 factors:
 1. light source
 2. surface
 3. observers

- 3 Types of Light source

1. Point sources

- Point sources emit light from a single point in all directions.
- The intensity of the point light decreasing with distance.

2. Parallel sources

Parallel sources can be considered as a point source which is far from the surface.

3. Distributed sources

Rays originates from a finite area.

- Reflection

When the light falls on the surface, the light gets absorbed and reflected. And the surface structure decides the amount of reflection and absorbed.

The observer's position and sensor spectrum sensitivities also affect the lightning effect.

- Diffuse reflection 漫反射

Diffuse reflection occurs on the surface which are rough.

$$I = I_p K_d \cos(\theta) = I_p K_d (N' \cdot L')$$

1. I 是漫反射光照强度
2. I_p 是光源光照强度
3. K_d 是漫反射系数, 从 0 到 1 for each RGB
4. N' 是 normalized surface, 物体表面某点的单位法向量
5. L' 是指向光源的 normalized direction 单位向量。

- Spectrum reflection 镜面反射

Spectrum reflection occurs on when light falls on shiny surface.

Phone Model

$$I = I_p \cos^n(a) W(\theta)$$

1. n 是 specular-reflection exponent
2. a 是观测者视线与反射光线的夹角

3. $W(\theta)$ 是镜面反射系数

– Ambient 环境光

Ambient is non-directional light source and not affected by anything in the world.

$$I = I_a K_a$$

1. I_a 是环境光强度

2. K_a 是环境光反射系数

- Shading model

Shading refers to the process of altering the color of the object in 3D scene. 影响着色效果的因素: the surface's angle to the light, the distance from lights, its angle to the camera and material property.

由于我们经常使用大量的多边形去模拟曲面, 因此我们希望相邻的多边形颜色可以平滑地过渡。

1. Flat shading 平面着色

In flat shading, all points over the surface of the polygon are displayed with the same intensity value.

2. Color interpolation shading/ Goraud shading

如果我们知道两个点的 intensity 那么我们能够知道这两个点之间任何一个点的强度。

3. Normal interpolation shading/ Phong shading

颜色差值着色需要知道多边形每个顶点的法向量, 时间成本比线性差值更高。